DVWA Vulnerable Web Application Lab
Report

1. Introduction

This report presents a structured web application security lab completed using Damn Vulnerable Web Application (DVWA) in a controlled local environment. The goal of the project was to understand how common web vulnerabilities are discovered, exploited, observed in network traffic, and explained from a defensive perspective. The lab followed a practical workflow that mirrors a simplified penetration testing cycle: environment setup, reconnaissance, exploitation, traffic inspection, vulnerability scanning, and remediation planning.

The project is especially relevant to demonstrate both technical execution and security reasoning. Rather than listing tools alone, the report explains what each tool revealed, why the findings matter, and how an organization should respond.

2. Lab Environment

- Operating System: Kali Linux running in Oracle VirtualBox
- Application Under Test: DVWA (Damn Vulnerable Web Application)
- Deployment Method: Docker container
- Target URL: http://localhost:8080
- Testing Tools: Nmap, Wireshark, OWASP ZAP
- DVWA Security Setting: LOW, to safely demonstrate vulnerabilities in a lab

Using Docker was valuable because it created an isolated and repeatable environment. This reduced the risk of affecting the host operating system and allowed the vulnerable application to be started, stopped, and reset quickly during testing.

Figure 1. Docker container running and DVWA reachable on localhost:8080.

3. Phase 1 - Environment Setup

The first phase focused on preparing a safe testing environment. Docker was installed and verified, the DVWA image was pulled, and the application was launched with port mapping from container port 80 to host port 8080. After opening the application in a browser, the default credentials were used, and the security level was set to LOW. This configuration ensured the vulnerable modules would behave as expected during the lab.

From a security perspective, this stage reflects a common real-world scenario: an exposed web application service running on a known port. Even before active testing begins, the existence of a reachable service creates an attack surface that an adversary can profile.

- Verified Docker service availability
- Pulled a prebuilt DVWA image to avoid local building issues
- Started the container and confirmed the mapping to port 8080
- Logged in to DVWA and configured the application for testing

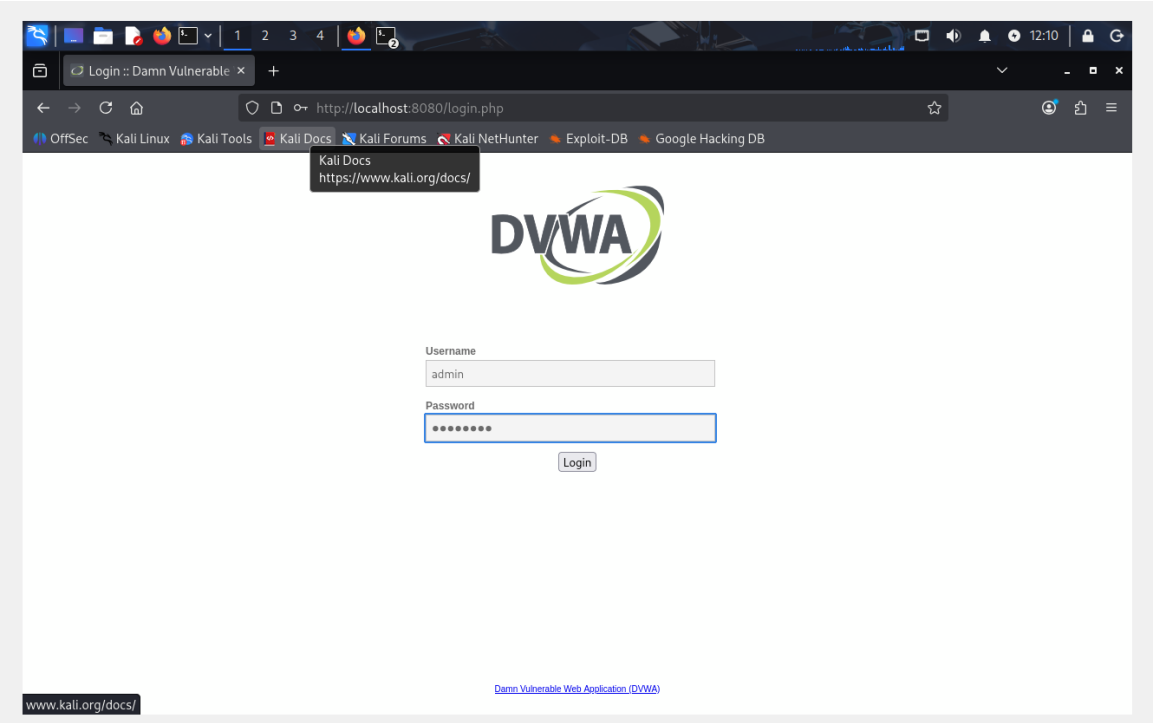

Figure 2. Output of docker ps showing the DVWA container and port mapping.

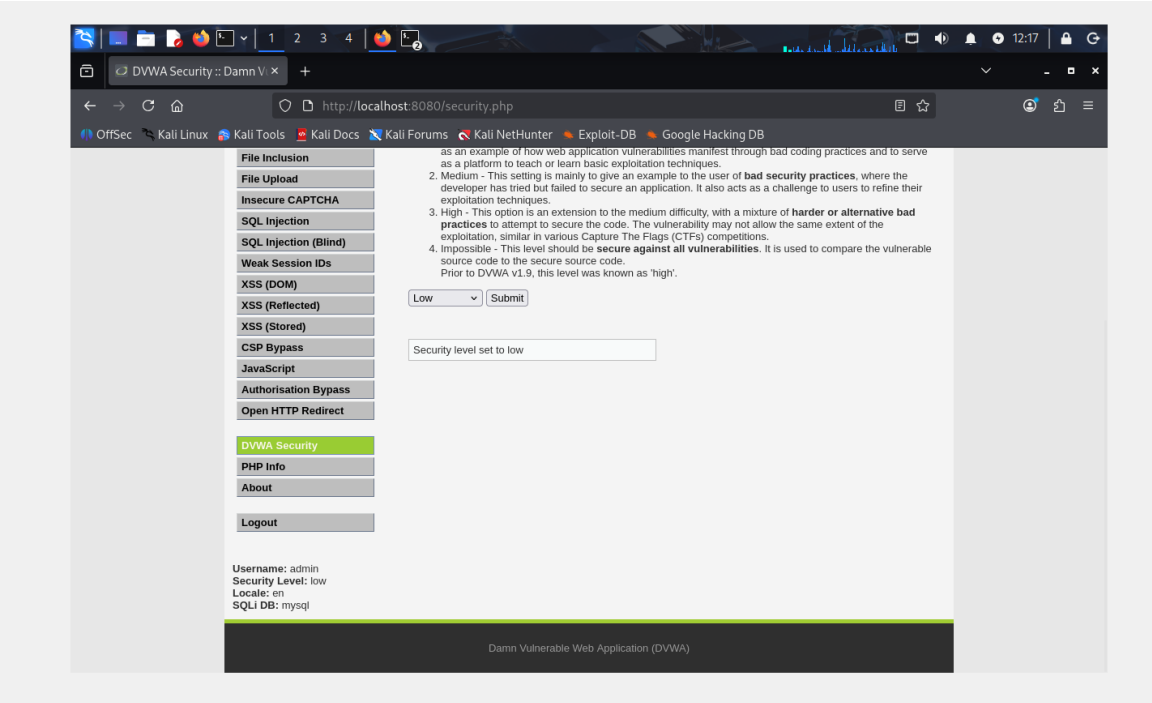

Figure 3. DVWA login page and security level configured to LOW.

4. Phase 2 - Reconnaissance with Nmap

The reconnaissance phase used Nmap to identify accessible services on the local target system. The command nmap -sV 127.0.0.1 was executed to detect open ports and gather service version information. The scan revealed that port 8080/TCP was open and serving HTTP through Apache.

This result is important because reconnaissance is often the first step in a real attack chain. An attacker who discovers an exposed HTTP service can begin testing for web-specific weaknesses such as injection flaws, insecure headers, weak authentication controls, or outdated server software. Service detection is also useful because it may help map the system to known vulnerabilities or misconfigurations.

It identified the attack surface and gave enough information to prioritize the next phase of testing.

- Open port discovered: 8080/TCP
- Detected service: HTTP
- Observed technology: Apache web server

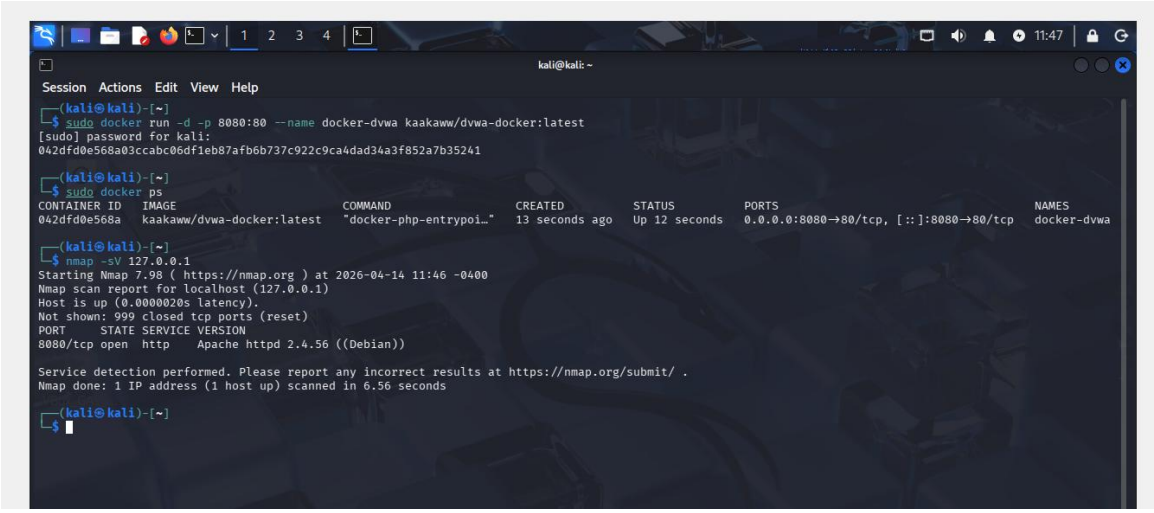

Figure 4. Nmap output showing port 8080 open with the Apache HTTP service identified.

5. Phase 3 - SQL Injection Testing and Exploitation

The SQL Injection module in DVWA was then tested to evaluate whether user input was being safely handled by the application. A normal request using a simple user ID returned a single expected record. After that, SQL injection payloads were introduced, including ' OR 1=1 -- and ' OR '1'='1. These payloads altered the intended logic of the SQL query, causing the application to return multiple records.

Technically, vulnerability exists because the application incorporates user-supplied input directly into a database query without proper sanitization or parameterization. Instead of treating the input solely as data, the application allows it to alter query logic. This is one of the most classic and serious web application vulnerabilities because it can lead to unauthorized data access, authentication bypass, record manipulation, and, in some cases, full database compromise.

This phase is strong because

- Baseline test returned one valid user record
- Injection payloads caused the application to return multiple records
- The behavior confirmed improper handling of database input

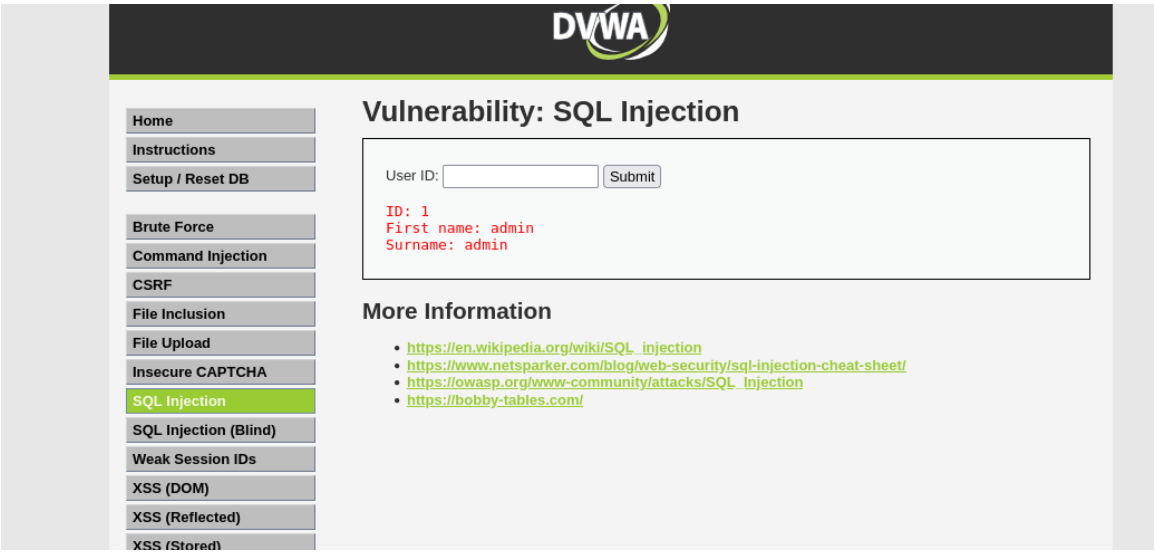

Figure 5. DVWA SQL Injection page showing a normal lookup that returns one expected record.

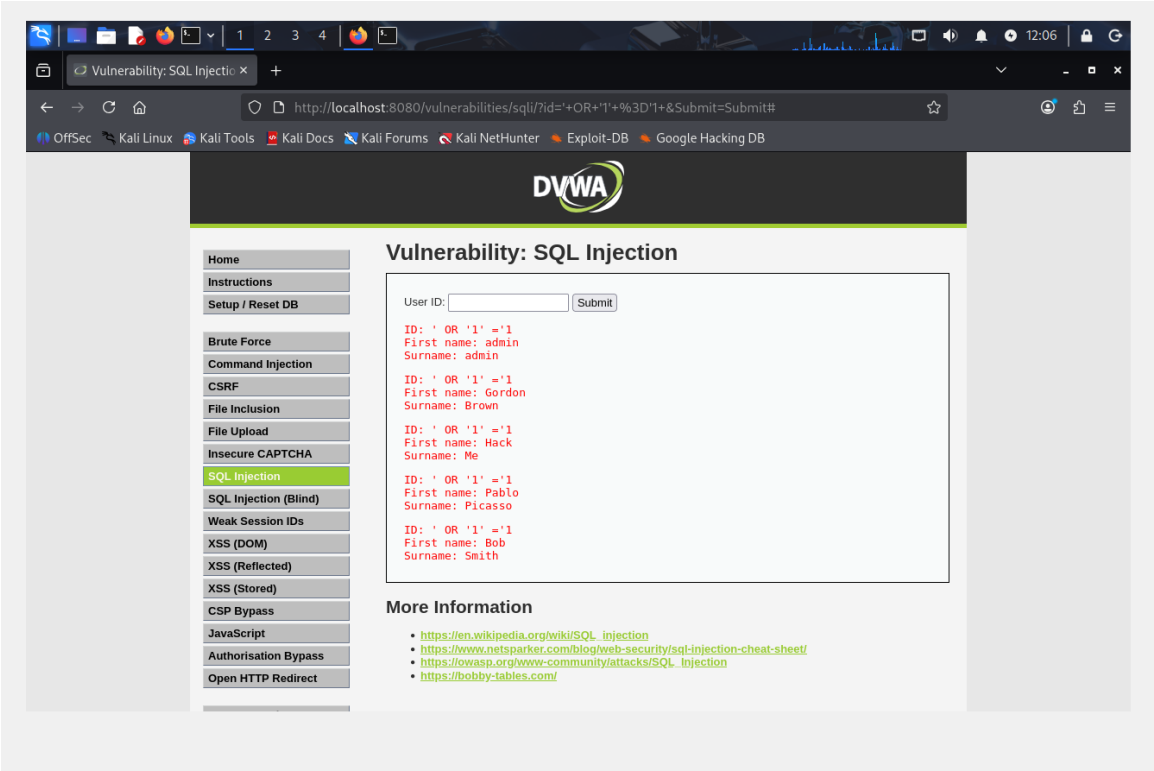

Figure 6. DVWA SQL Injection page showing multiple records returned after the injection payload.

6. Phase 4 - Traffic Analysis with Wireshark

Wireshark was used to capture HTTP traffic on the loopback interface while interacting with DVWA. The packet capture showed login activity and requests to vulnerable pages, including visible parameters such as id=1. Because the application was using HTTP rather than HTTPS, the traffic was transmitted in plaintext.

This observation is significant because encryption protects data in transit. Without HTTPS, an attacker positioned on the same network could inspect requests, capture credentials, observe session identifiers, or replay manipulated requests. Even when the application logic is the focus of testing, transport security still matters because weak transmission security increases overall exposure.

This section helps explain that security is layered. A system may have an application flaw, such as SQL injection, but it may also have transport-level weaknesses that make attacks easier or increase the damage caused by interception.

- Observed HTTP GET and application requests in plaintext
- Visible request parameters within packet data
- Confirmed lack of transport encryption

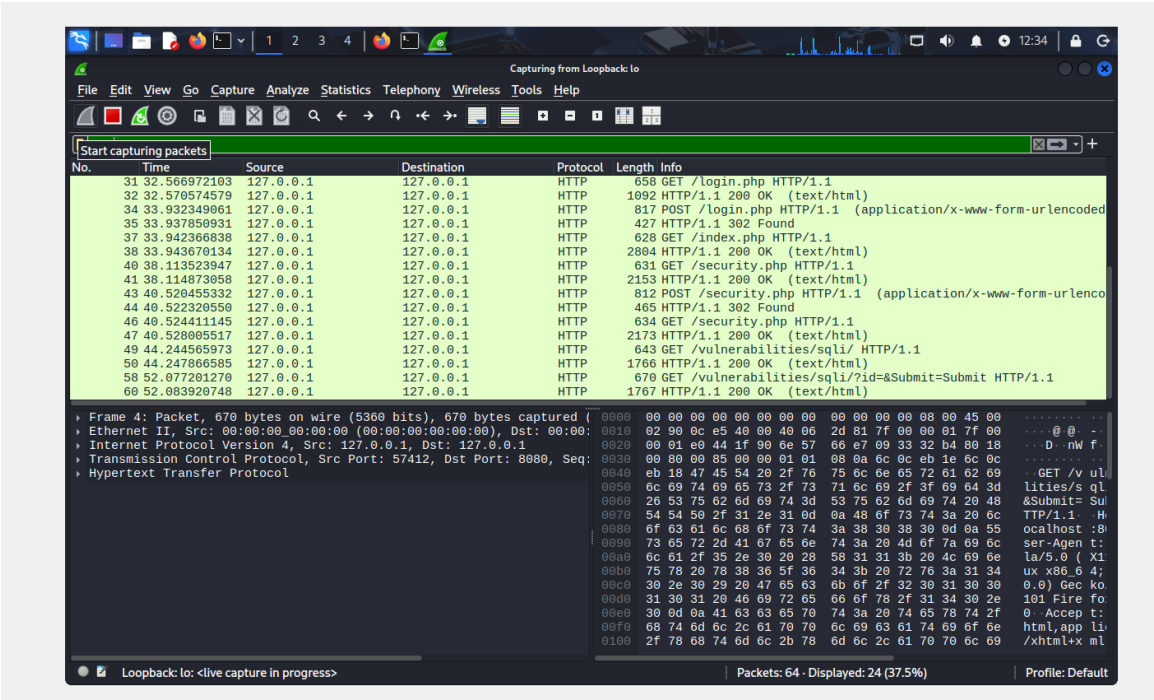

Figure 7. Wireshark packet capture shows HTTP traffic and readable request details.

Suggested optional extra screenshot: include a close-up of the Wireshark packet detail pane, highlighting the HTTP method, URI, and parameter field. This makes your explanation much stronger during a presentation.

7. Phase 5 - Vulnerability Scanning with OWASP ZAP

OWASP ZAP was used to perform automated scanning against the DVWA environment. The scan generated 13 alerts across medium, low, and informational severity levels. The findings included insecure cookie settings, missing security headers, and indicators of information disclosure.

These issues matter because they show that security weaknesses are not limited to one dramatic flaw. Real applications often contain several smaller configuration problems that, when combined, increase risk. For example, missing HttpOnly or SameSite cookie attributes can make session theft

easier, while absent headers such as Content-Security-Policy or X-Frame-Options reduce protection against browser-based attacks like clickjacking or some client-side injection scenarios.

This phase also demonstrates good judgment: automated tools are useful for coverage and speed, but their results still require analyst interpretation. I found the following:

- Total alerts observed: 13
- Medium-risk items included cookie security weaknesses
- Low-risk items included missing security headers and information disclosure
- Informational items provided additional context about the application structure

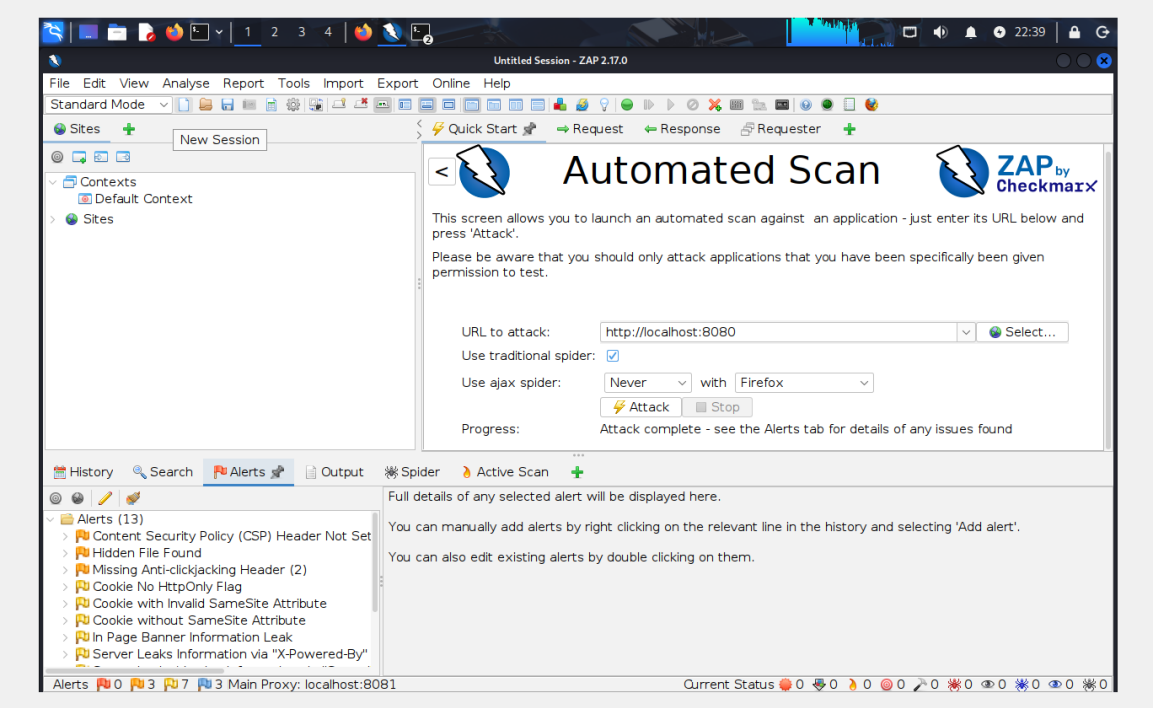

Figure 8. ZAP results page showing the alert count and severity breakdown.

8. Key Findings

- DVWA was accessible through an exposed HTTP service on port 8080
- The application was vulnerable to SQL injection because input was not safely handled
- Traffic analysis confirmed that requests were transmitted in plaintext over HTTP
- Automated scanning identified additional security weaknesses related to cookies, headers, and information disclosure
- The lab demonstrated how reconnaissance, exploitation, and analysis connect within one attack workflow